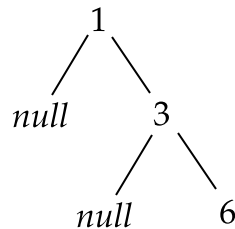


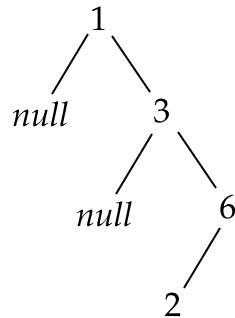
שאלה 1

א. הטענה לא נכונה.

הסדרה 1, 3, 6, 2, 4, 5 לא יכולה להיות סריקת *preorder* של עץ חיפוש בינארי.
 בסריקת *preorder* מבקרים את השורש, אח"כ את תת העץ השמאלי ואחרי זה את תת העץ הימני.
 1 הוא שורש העץ.
 $3 > 1$, לכן יורדים לתת העץ הימני בעל שורש 3.
 $6 > 3$, לכן יורדים לתת העץ הימני בעל שורש 6.
 עד כה העץ בהכרח נראה כך:



כעת $2 < 6$, לכן יורדים לתת העץ השמאלי, כלומר נקבל:



אבל זה לא עץ חיפוש בינארי, מאחר ואם ננסה למצוא את 2 נקבל את המסלול $1 \rightarrow 3 \rightarrow \text{null}$.

ב. הטענה לא נכונה.

ככל שנקטין את ההסתברות של הוספת צומת ברמה מעל תיווצרנה בממוצע פחות רמות במבנה.
 במקרה הזה ההסתברות לקבל 0 ולהוסיף צומת חדש מעל הרמה הנוכחית היא $\frac{1}{10}$.
 נסתכל על תוחלת מספר ההטלות הדרוש עד שמקבלים 1 המסמלת את מספר הרמות ברשימה כפי שנלמד בהרצאה:

$$\mathbb{E}_{p=\frac{1}{2}} = \frac{1}{1 - \frac{1}{2}} = 2$$

$$\mathbb{E}_{p=\frac{1}{10}} = \frac{1}{1 - \frac{1}{10}} = \frac{1}{\frac{9}{10}} = \frac{10}{9} \approx 1.111$$

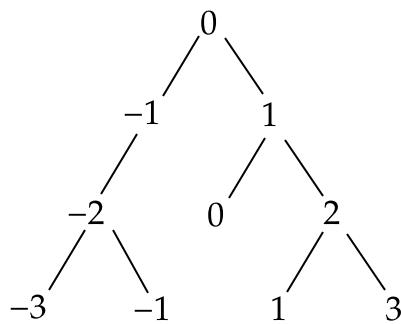
$$\mathbb{E}_{p=\frac{1}{10}} = 1.111 < 2 = \mathbb{E}_{p=\frac{1}{2}}$$

ג. הטענה נכונה.

נגדיר דרגה $r(x)$ של צומת כלשהו בעץ באופן הבא:

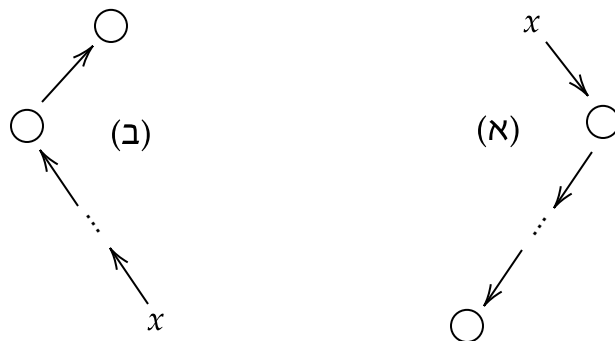
- דרגת השורש היא 0.
- אם דרגת הצומת היא k אז דרגת בנו השמאלי היא $k - 1$ ודרגת בנו הימני היא $k + 1$.

לדוגמה:



תהי סדרה של m פעולות $\text{succ}(x)$. עבור $i \in [m]$ נסמן את זמן הריצה של הפעולה ה- i ב- t_i . נסמן x_0 הצומת המינימלי שממנו מתחילים ו- x_i הצומת שאליו הגענו לאחר i פעולות $\text{succ}(x)$. נשתמש בשיטת הפוטנציאל ונגדיר $\phi_i = r(x_i)$.

נבחין כי בסיור *inorder* יש שני מצבים שונים בהם יידרש יותר מצעד אחד בעץ על מנת להגיע לאיבר העוקב:



המחיר המשוערך של מקרה א' מורכב מירידה אחת ימינה וירידה אחת או יותר שמאלה

המחיר המשוערך של ירידה ימינה:

$$a_{right}^{down} = t_{right}^{down} + \overbrace{\phi(x_i \rightarrow down\ right)}^{r(x_i)+1} - \overbrace{\phi(x_i)}^{r(x_i)} = 1 + r(x_i) + 1 - r(x_i) = 2$$

המחיר המשוערך של ירידה שמאלה:

$$a_{left}^{down} = t_{left}^{down} + \overbrace{\phi(x_i \rightarrow down\ left)}^{r(x_i)-1} - \overbrace{\phi(x_i)}^{r(x_i)} = 1 + r(x_i) - 1 - r(x_i) = 0$$

לכן במקרה הזה המחיר המשוערך של ההגעה לאיבר העוקב הוא 2.

המחיר המשוערך של מקרה ב' מורכב מעליה אחת או יותר שמאלה ועליה אחת ימינה

המחיר המשוערך של עליה שמאלה:

$$a_{left}^{up} = t_{left}^{up} + \overbrace{\phi(x_i \rightarrow up\ left)}^{r(x_i)-1} - \overbrace{\phi(x_i)}^{r(x_i)} = 1 + r(x_i) - 1 - r(x_i) = 0$$

המחיר המשוערך של ירידה ימינה:

$$a_{right}^{up} = t_{right}^{up} + \overbrace{\phi(x_i \rightarrow up\ right)}^{r(x_i)+1} - \overbrace{\phi(x_i)}^{r(x_i)} = 1 + r(x_i) + 1 - r(x_i) = 2$$

לכן גם במקרה הזה המחיר המשוערך של ההגעה לאיבר העוקב הוא 2, כלומר בסה"כ:

$$\sum_{i=1}^m a_i \leq 2m$$

נשים לב כי $\phi_0 = r(x_0) = \min_{x \in Tree} r(x)$, לכן $\phi_m \geq \phi_0$. לפי כך מתקיים:

$$\sum_{i=1}^m a_i = \sum_{i=1}^m t_i + \phi_m - \phi_0$$

$$\Rightarrow \sum_{i=1}^m t_i = \sum_{i=1}^m a_i + \underbrace{\phi_0 - \phi_m}_{\text{ביטוי אי חיובי}} \leq \sum_{i=1}^m a_i \leq 2m$$

לכן סיבוכיות הזמן המשוערכת של פעולת העוקב $\text{succ}(x)$ היא $O(1)$ כנדרש.

ד. הטענה נכונה.

נבצע את השינויים הבאים:

- נשמור בכל צומת מצביע לצומת האב שלו. בצמתים של "השרוך השמאלי" של העץ לצורך הסבר האלגוריתם שנבצע נסמן את צומת האב x_{i+1} עבור צומת x_i .
- נשמור בשורש מצביע לצומת עם האיבר המינימלי של העץ, נסמנו x_0 . בכל פעולת הכנסה נבדוק אם האיבר החדש קטן מ- x_0 ונעדכן את השדה הזה בהתאם. במקרה של הוצאת x_0 נדאג למצוא את האיבר המינימלי הבא, מה שניתן לעשות בשני צעדים בלבד, לכן סיבוכיות הזמן של פעולות ההכנסה וההוצאה נשארת $O(\log n)$.
- נשמור בכל צומת את האיבר המקסימלי בתת העץ שלו, נסמנו $M(x_i)$. בכל פעולת הכנסה והוצאה נעדכן את השדה הזה להיות המקסימום בין האיבר המקסימלי בתת העץ של הבן השמאלי לאיבר המקסימלי בתת העץ של הבן הימני לאחר שהכנסנו/הוצאנו. עדכון זה מוסיף השוואה אחת לצעדים הרגילים של פעולות ההכנסה וההוצאה, לכן, מאחר ובוצעו שינויים רק במסלול ההכנסה/ההוצאה, סיבוכיות הזמן של פעולות ההכנסה וההוצאה נשארת $O(\log n)$.

האלגוריתם:

נסמן את האיבר שאנחנו מחפשים ב- t .
 נתחיל מ- x_0 (ניגש אליו ב- $O(1)$, כי שמרנו מצביע אליו בשורש). אם $t = x_0$, אז סיימנו. אחרת נעלה לצומת האב שלו, x_1 . נרצה לבדוק אם האיבר שאנחנו מחפשים נמצא בתת העץ שלו, לכן נבדוק אם $t \leq M(x_1)$. אם כן - נבצע חיפוש רגיל בתת העץ של x_1 , אחרת נעלה ל- x_2 .
 באופן דומה: אם $t \leq M(x_i)$, אז נבצע חיפוש רגיל בתת העץ של x_i , אחרת נעלה ל- x_{i+1} . נמשיך כך עד שנגיע לשורש.

סיבוכיות הזמן:

לפי טענה מההרצאה מספר העלים L בעץ $2 - 3$ מקיים $2^h \leq L \leq 3^h$ כאשר h הוא גובה העץ. ביצענו i צעדים ונעצרנו בצומת x_i , לכן גובה תת העץ שלו הוא i ולפי כך מספר העלים שלו $2^i \leq L_i \leq 3^i$. דרגת t היא לכל היותר L_i , כי היא חסומה מלמעלה על ידי כמות העלים, לכן $2^i \leq L_i \leq 3^i$. כמו כן, מאחר והגענו לצומת x_i , t בהכרח לא נמצא בתת העץ השמאלי של x_i (אחרת היינו נעצרים ב- x_{i-1}), לכן

$$d \geq L_{i-1} + 1 \geq 2^{i-1} + 1$$

לכן $2^{i-1} + 1 \leq d \leq 3^i$ אז מתקיים:

$$2^{i-1} + 1 \leq d \implies i-1 \leq \log_2 d \stackrel{\text{אסימפטוטית}}{\implies} i \leq \log_2 d$$

$$d \leq 3^i \implies \log_3 d \leq i$$

$\Downarrow$

$$\log_3 d \leq i \leq \log_2 d$$

$\Downarrow$

$$i = O(\log d)$$

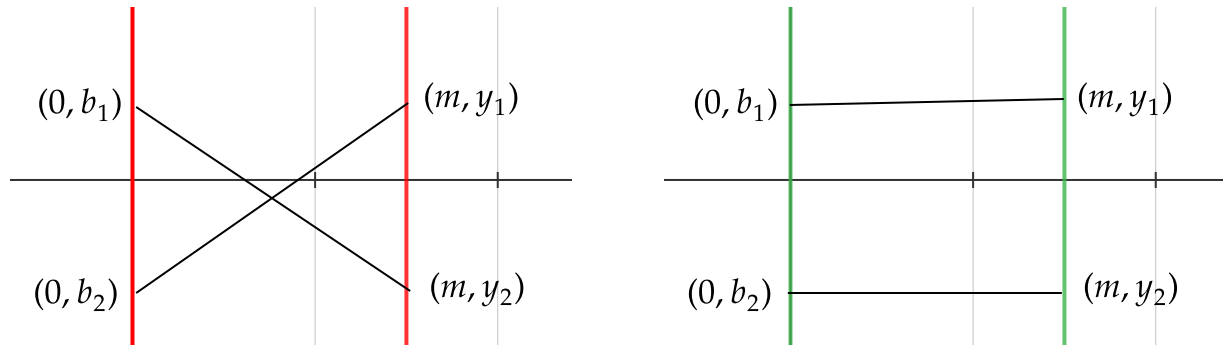
לקח לנו i פעולות כדי להגיע לצומת x_i , חיפוש רגיל בעץ בגובה i לוקח $O(i)$ פעולות, לכן:

$$i + O(i) = O(i) = O(\log d)$$

כלומר בסה"כ נבצע $O(\log d)$ פעולות כנדרש.

שאלה 2

לצורך המימוש נשתמש בעץ AVL. בכל צומת נשמור שלושה שדות a, b ו- y כאשר העץ ממין לפי b . נאתחל את y ע"י $am + b$, כלומר ערך הקו בנקודה $x = m$. נשים לב ש- b זה ערך הקו בנקודה $x = 0$. שני קווים 1, 2 לא נחתכים אם מתקיים $b_1 > b_2 \wedge y_1 > y_2$ (או $b_1 < b_2 \wedge y_1 < y_2$), להלן ציורים להמחשה:



$Init(m)$: מאתחלת counter עם ערך התחלתי 0 ועץ AVL ריק.

סיבוכיות זמן: $O(1)$ כנדרש.

$InsertLine((a, b))$: נאנדקס את שדות הקו החדש ב-1 ואת שדות הקו שמשווים אליו ב-2. נחפש את המיקום הנכון עבור הקו החדש לפי b ולאורך המסלול נוודא שבכל צומת מתקיים $b_1 > b_2 \wedge y_1 > y_2$ (או $b_1 < b_2 \wedge y_1 < y_2$). אם הבדיקה צלחה בצומת כלשהו אז אנחנו יודעים שהקו לא נחתך עם הקו של אותו הצומת. נניח בה"כ שפנינו ימינה, אז הקו לא נחתך גם עם אף קו בתת העץ השמאלי של אותו הצומת, שכן כל הקווים שתת העץ השמאלי מכיל נמצאים מתחת לקו של הצומת. אם הצלחנו להגיע למיקום הנכון ולעבור את כל הבדיקות בדרך נכניס את הקו החדש לעץ ובבצע $counter++$.

סיבוכיות זמן: אנו מוצאים את המיקום הנכון עבור האיבר החדש. בכל צעד אנו מבצעים לכל היותר 3 פעולות השוואה $b_1 > b_2, b_1 > b_2 \wedge y_1 > y_2, b_1 < b_2 \wedge y_1 < y_2$ ב- $O(1)$. בדומה לפעולת $insert$ הרגילה של עץ AVL ייקח לנו לכל היותר $\log n$ צעדים עד שנמצא את המיקום הנכון. בסה"כ כולל פעולת ההכנסה נבצע לכל היותר $3 \log n + 1$ פעולות, לכן הפעולה $InsertLine$ היא $O(\log n)$ כנדרש.

$Size()$: נחזיר את $counter$.

סיבוכיות זמן: $O(1)$ כנדרש.

שאלה 3

לצורך המימוש נשתמש ב-*counter* אשר מסמל את כמות נקודות העניין שהוספנו, במערך דינאמי *points* מטיפוס *bool* כאשר האינדקסים שלו מסמלים את נקודות העניין והערך מסמל האם הכנסנו את הנקודה, ב-*current* אשר מסמל את המיקום הנוכחי של הרובוט במערך וב-*max* שמסמן את הערך המותר המקסימלי לפי ההגבלה.

ניהול המערך:

תחילה נקצה מערך בגודל 2. באופן רקורסיבי נסמן את גודלו ב- k ולאחר שמתקיים $counter == \frac{k}{2}$ נקצה מערך חדש בגודל $2k$ ונעתיק אליו את כל האיברים. נשים לב כי לאחר הכנסת n נקודות כמות הפעמים שהמערך גדל חסומה מלמעלה על ידי $\lceil \log 2n \rceil$ $(2^k \leq 2n \implies k \leq \log 2n)$.

Init(): נקצה *counter*, *current* עם ערך התחלתי 0, *max* עם ערך התחלתי 2 ו-*points* בגודל 2 עם הערך *false*.

סיבוכיות הזמן:

הקצאת מערך מתבצעת ב- $O(1)$ כפי שנלמד בהרצאה, לכן הסיבוכיות הכוללת היא $O(1)$.

AddPoint(i):

נגדיל את גודל המערך במידת הצורך, אחר כך ניגש ל-*points[i]*, נשנה אותו ל-*true*, נבצע $counter++$ וגם $max = 2 \cdot counter$.

MoveToPoint():

נקדם את *current* עד שנגיע ל- $max + 1$ ואז נעצור בלי לעשות כלום או עד שנגיע לתא הראשון במערך עם *true* ואז נדפיס את האינדקס שלו.

סיבוכיות הזמן המשוערכת של שתי הפעולות יחד:

נשתמש בשיטת הצבירה:

תהי סדרת M פעולות כלשהי המורכבת מהפעולות הבאות:

- l פעולות *AddPoint*
- m פעולות *MoveToPoint* כך שבקריאה ה- $m \geq i \geq 1$ מקדמים את *current* t_i פעמים

הוספנו l נקודות, לכן, לפי ההגבלה הנתונה, כל m הפעולות $MoveToPoint$ יוכלו לקדם את $current$ לכל היותר $2l$ פעמים, כלומר:

$$\sum_{i=1}^m t_i \leq 2l$$

כאמור המערך לא גדל יותר מ- $\lceil \log 2l \rceil$ פעמים, אז כמות פעולות ההעתקה שבוצעו חסומה:

$$\sum_{k=1}^{\lceil \log 2l \rceil} 2^k = 2(2^{\lceil \log 2l \rceil} - 1) \leq 4l$$

לפי כך מתקיים:

$$Total\ Time = \overbrace{O(1 + 4l)}^{AddPoint} + \overbrace{\sum_{i=1}^m O(1 + t_i)}^{MoveToPoint} \leq O(l) + O(m + 2l) = O(l + m) = O(M)$$

לכן סיבוכיות הזמן המשוערכת היא $O(1)$ כנדרש.

שאלה 4

נשתמש בשלושה עצי $Trie$, $both$ עבור כל צמדי המילים כולל $-$, $first$ עבור המילים הראשונות בצמדים ו- $second$ עבור המילים השניות בצמדים. בשני העצים $first$ ו- $second$ נשמור בכל צומת $wordCounter$ עם ערך התחלתי 1 שעוקב אחרי כמות המילים השונות החל מהאות של הצומת. כאשר נכניס מילה חדשה נבצע $wordCounter++$ בכל אחד מצמתי מסלול ההכנסה.

$Init(m)$: נאתחל את שלושת העצים ונשמור את m .

סיבוכיות הזמן: $O(1)$ כנדרש.

$Insert(w_1 - w_2)$:

נוודא שמחרוזת זו טרם הוכנסה, נכניס ל- $both$ את המחרוזת $w_1 - w_2$, ל- $first$ את w_1 ול- $second$ את w_2 . כאמור נבצע $wordCounter++$ בכל אחד מצמתי מסלול ההכנסה.

סיבוכיות הזמן:

סיבוכיות הזמן של הפעולה היא $O(|w_1 - w_2|) + O(|w_1|) + O(|w_2|)$ לפי סיבוכיות פעולת ההכנסה כפי שנלמד בהרצאה, כלומר בסה"כ $O(|w_1| + |w_2|)$ כנדרש.

$Magic(w_1 - w_2)$:

אם המחרוזת לא נמצאת ב- $both$ נחזיר $false$.

נאתחל $magicCounter1$ עם ערך התחלתי 0.

נעבור על w_1 ב- $first$, עבור כל צומת נעבור על כל האותיות ב- Σ שקטנות ממש לקסיקוגרפית מהצומת, נבדוק אם קיימות מילים שמתחילות מהאותיות האלה ואם כן אז נוסיף את כל ה- $wordCounters$ שלהם ל- $magicCounter1$.

לאחר שהגענו לסוף w_1 נחזיר $false$ אם $magicCounter1 < m$, אחרת נמשיך ל- $second$.

נאתחל $magicCounter2$ עם ערך התחלתי 0.

נעבור על w_2 ב- $second$, עבור כל צומת נעבור על כל האותיות ב- Σ שגדולות לקסיקוגרפית מהצומת, נבדוק אם קיימות מילים שמתחילות מהאותיות האלה ואם כן אז נוסיף את כל ה- $wordCounters$ שלהם ל- $magicCounter2$. לאחר שהגענו לסוף w_1 נחזיר $false$ אם $magicCounter2 < m$, אחרת נחזיר $true$.

סיבוכיות הזמן:

במעבר על $both$ אנו מבצעים לכל היותר $|w_1| + |w_2|$ פעולות, במעבר על w_1 לכל היותר $|\Sigma| \cdot |w_1|$ פעולות ובמעבר על w_2 לכל היותר $|\Sigma| \cdot |w_2|$, כלומר בסה"כ ביצענו $(|\Sigma| + 1)(|w_1| + |w_2|)$ פעולות, לכן סיבוכיות הזמן היא $O(|w_1| + |w_2|)$ כנדרש.